

Sistemas de Numeração Assimétrica (ANS)

Codificação Entrópica de Alta Performance, Modelagem de Contexto 1D/2D e Fatiamento de Planos de Bits

Área: Teoria da Informação & Compressão

Algoritmos: tANS (C) & rANS Streaming (Python)

Status: 100% Validado (Lossless)

Data: Agosto / 2026

SUMÁRIO EXECUTIVO DO PROJETO:

Este projeto implementa uma suíte completa de compressão de dados baseada em **Asymmetric Numeral Systems (ANS)**, tecnologia de ponta criada por Jarek Duda que revolucionou a codificação entrópica mundial (adotada no *Zstandard/Facebook*, *JPEG XL* e *Apple LZFSE*). O projeto integra duas frentes: uma implementação estática de alta velocidade em **linguagem C (tANS)** baseada em autômatos finitos e tabelas de busca reversa (RLT), e um codec moderno e adaptativo em **Python (rANS Streaming)** com **Estado Contínuo**, **Predição Contextual Espacial 2D**, **Lookahead 1D** e decomposição de imagens em **8 Planos de Bits**.

Resultados Chave: Taxas de compressão de até **85.82%** em sequências 1D e até **78.03%** em planos estruturais de imagens, com **100% de integridade exata (reversibilidade perfeita)** comprovada em todos os testes.

1. Introdução e Fundamentos Teóricos do ANS

Na teoria clássica da compressão de dados, os desenvolvedores enfrentavam um dilema histórico entre duas abordagens entrópicas:

- Codificação de Huffman:** Extremamente rápida devido ao uso de tabelas de prefixo pré-calculadas, porém ineficiente para distribuições onde as probabilidades exigem quantidades fracionárias de bits por símbolo (atribui no mínimo 1 bit por símbolo, limitando a compressão em dados muito redundantes).
- Codificação Aritmética (AC):** Matematicamente ótima, atingindo o limite de Shannon ao operar com precisão de frações de bits, contudo computacionalmente pesada devido a sucessivas divisões e multiplicações de alta precisão em cada símbolo codificado.

O **Asymmetric Numeral Systems (ANS)**, formulado pelo matemático polonês **Jarek Duda**, solucionou esse dilema unificando o melhor dos dois mundos: alcança a **velocidade de processamento de Huffman** através de operações de deslocamento de bits e tabelas de transição de estado simples, enquanto atinge a **eficiência de compressão da Codificação Aritmética**, permitindo frações de bits por símbolo.

Algoritmo	Velocidade	Taxa de Compressão	Bits Fracionários	Aplicações Reais
Huffman	Ultra Rápida (Tabelas)	Moderada (Subótima)	Não (≥ 1 bit/símbolo)	DEFLATE, Gzip, JPEG clássico
Codificação Aritmética	Lenta (Divisões caras)	Ótima (Limite de Shannon)	Sim (Perfeito)	H.264/CABAC, JPEG 2000
tANS (Table ANS)	Extremamente Rápida	Quase Ótima (~Shannon)	Sim (via FSM estática)	Zstandard (Meta), LZFSE (Apple)
rANS (Range ANS)	Muito Rápida (Aritmética leve)	Ótima + Adaptativa	Sim (via estado contínuo)	JPEG XL, Oodle (Epic Games)

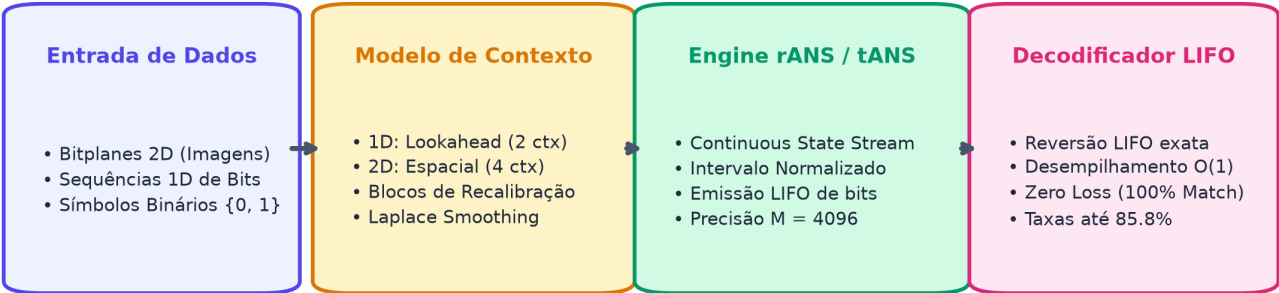
2. Arquitetura do Sistema e Estrutura dos Arquivos

O projeto foi organizado de forma modular para permitir tanto a avaliação de baixo nível em linguagem C quanto o processamento flexível de imagens e modelagem estatística 2D em Python.

Arquivo / Componente	Linguagem	Função Principal e Responsabilidade no Pipeline
Encoder_Ans.c	C (Nativo)	Codificador tANS estático de alta velocidade com tabelas pré-computadas (RANGE=8, CONFIG=6) e emissão de bitstream alinhado a bytes.
Decoder.c	C (Nativo)	Decodificador tANS com montagem dinâmica de tabela reversa (RLT), leitura LSB/MSB e desempilhamento simétrico de símbolos.

ANS-Streaming Encoder.py	Python 3	Codificador rANS com Estado Contínuo, suporte a contextos 1D e 2D espacial, divisão de janelas de recalibração e serialização compacta em texto/binário.
ANS-Streaming-Decoder.py	Python 3	Decodificador rANS simétrico com otimização collections.deque para desempilhamento $O(1)$ e verificação estrita de identidade lossless.
context_model.py	Python 3	Módulo estatístico: gerencia modelos de contexto 1D (2 estados) e 2D (4 estados), normalização com precisão de 12 bits ($M=4096$) e cabeçalhos hexadecimais.
Bit_Planes.py	Python / OpenCV	Fatiamento de imagens 8-bit em 8 planos de bits booleanos (0 a 7), salvamento matricial e geração do gráfico comparativo de visualização.
run_all.py	Python 3	Suíte completa de benchmarking automatizado. Executa encode/decode em todos os inputs 1D e planos 2D, calculando overheads, bits reais, taxas e tempos.

Arquitetura e Fluxo de Dados do Pipeline de Compressão ANS



3. Implementação tANS Estática em C (Table-based)

A implementação em C fundamenta-se no **tANS (Table-based ANS)**. Neste modelo, a aritmética de transição é pré-computada em tempo de compilação ou inicialização em matrizes estáticas, transformando o compressor em um **Autômato Finito Determinístico (FSM)** extremamente eficiente.

Mecânica do Codificador (Encoder_Ans.c):

- Parâmetros do Sistema:** Configurado com $RANGE = 8$ e $CONFIG = 6$ para o alfabeto binário {0, 1}.
- Transição de Estado:** O estado atual $state \in [8, 15]$ é mapeado pelo índice $state_idx = state - 8$. As tabelas A_TABLE e B_TABLE fornecem diretamente o novo estado.
- Normalização e Emissão:** As tabelas A_NBITS e B_NBITS indicam quantos bits (0, 1, 2 ou 3 bits) devem ser descarregados no buffer BitstreamWriter para manter o estado no intervalo válido.
- Flush Final:** Ao término da sequência, o offset final do estado ($state - RANGE$) é emitido em $FLUSH_BITS = 3$ bits, permitindo a reinicialização perfeita do decodificador.

Mecânica do Decodificador Reverso (Decoder.c):

- Construção da Reverse Lookup Table (RLT):** Na inicialização (função `build_rlt()`), o decodificador analisa as tabelas do codificador e constrói o mapeamento inverso exato: para cada estado de destino, determina o símbolo original correspondente, a quantidade de bits a consumir do bitstream e o estado-base.
- Comportamento LIFO (Pilha):** Como o ANS opera matematicamente como uma pilha (o último símbolo codificado é o primeiro decodificado), o decodificador consome o bitstream do final para o início, garantindo simetria perfeita.

```
ESTRUTURA DE DADOS DA TABELA REVERSA EM C (RLT):
typedef struct { int symbol; uint8_t nbits; uint8_t base_state; } DecodeEntry;
DecodeEntry g_rlt[RANGE]; // Mapeia diretamente: estado_idx -> (símbolo, nbits, base)
Transição de decodificação: state = entry->base_state + bitstream_read_bits_msb(reader, entry->nbits);
```

4. Implementação rANS Streaming e Modelagem Contextual v2 (Python)

A evolução do projeto implementou o **rANS (Range ANS) com Estado Contínuo** em Python, introduzindo predição estatística avançada baseada em cadeias de Markov 1D e relações espaciais 2D.

FORMULAÇÃO MATEMÁTICA FUNDAMENTAL DO rANS:

Dado um alfabeto com soma de frequências $M = 4096$ (precisão de 12 bits), frequência do símbolo f_s e frequência acumulada $F_s = \sum_{k < s} f_k$:

- Codificação (SCl_rANS):** $x' = \left\lfloor \frac{x}{f_s} \right\rfloor \cdot M + F_s + (x \bmod f_s)$
- Normalização do Encoder:** Enquanto $x \geq 2 \cdot f_s$, emite o bit menos significativo $(x \gg 1)$ e desloca $x \gg= 1$.
- Decodificação (SDl_rANS):** Encontra s onde $F_s \leq (x \bmod M) < F_{s+1}$, e restaura $x_{\text{prev}} = \left\lfloor \frac{x}{M} \right\rfloor \cdot f_s + (x \bmod M) - F_s$.
- Normalização do Decoder:** Enquanto $x_{\text{prev}} < M$, ingere bits do stream: $x_{\text{prev}} = (x_{\text{prev}} \ll 1) \mid \text{bit}$.

Inovações Arquiteturais da Versão Contextual:

- Estado Contínuo (Continuous State):** Em compressores ingênuos, o estado do codec é reinicializado a cada bloco curto, gerando enorme desperdício com bits de flush e alinhamento (padding). O modelo deste projeto mantém o macro-estado ativo continuamente ao longo de todos os milhares de símbolos, emitindo apenas bits excedentes quando necessário. Isso reduz o overhead de estado para apenas ~13 bits em todo o arquivo.
- Modelagem de Contexto 1D (Lookahead):** Para fluxos lineares, a probabilidade do símbolo s_i é condicionada ao valor do próximo símbolo s_{i+1} ($P(s_i \mid s_{i+1})$). Devido à decodificação reversa LIFO, quando o decodificador atinge a posição s_i , o símbolo s_{i+1} já foi decodificado, tornando o contexto perfeitamente conhecido sem necessidade de transmitir árvores de predição adicionais.
- Modelagem de Contexto Espacial 2D para Imagens:** Em matrizes bidimensionais com largura W , o pixel s_i é condicionado a dois vizinhos geométricos: o vizinho à direita ($r_d = \text{pixel}[i+1]$) e o vizinho inferior ($b_d = \text{pixel}[i+W]$). Isso gera **4 contextos estatísticos independentes** (00, 01, 10, 11), capturando bordas, texturas e gradientes espaciais com precisão.
- Recalibração Adaptativa por Janela (recalc_window):** O stream é pré-escaneado em blocos (ex: 1000 símbolos) para calcular tabelas de frequência locais normalizadas com suavização de Laplace ($f_0, f_1 \geq 1$). Cada tabela é codificada de forma ultracompacta em 12 bits por contexto no cabeçalho.
- Otimização de Performance com Deque $O(1)$:** A inversão do bitstream para decodificação em Python através de listas nativas gera custo $O(n^2)$ com `pop(0)`. A implementação utiliza `collections.deque` com `popleft()`, garantindo tempo amortizado $O(1)$ por operação e alta taxa de transferência.

5. Fatiamento de Planos de Bits (Bit-Plane Slicing)

A estratégia para compressão de imagens em escala de cinza consiste no **Bit-Plane Slicing**, implementado em `Bit_Planes.py`. Cada pixel de 8 bits (valores de 0 a 255) é decomposto em 8 matrizes binárias independentes através de máscaras bitwise (`pixel & (1 << k)`):

- Planos Superiores (Planos 7, 6, 5 - MSB):** Contêm as informações estruturais predominantes da imagem, áreas homogêneas (céu, pele, fundos uniformes) e alta correlação entre pixels adjacentes. Apresentam **baixíssima entropia**, permitindo que o rANS contextual alcance taxas de compressão expressivas (**até 78.03% no Plano 7**).
- Planos Inferiores (Planos 0, 1 - LSB):** Armazenam variações sutis e ruído do sensor de captura. Comportam-se como ruído estocástico puro com distribuição praticamente uniforme de 0s e 1s (entropia ≈ 1.0 bit/símbolo). Nesses planos, os dados são incompressíveis, e o pequeno overhead dos cabeçalhos resulta em taxas ligeiramente negativas (-2.81% no Plano 0), comportamento esperado e teoricamente perfeito segundo o limite de Shannon.

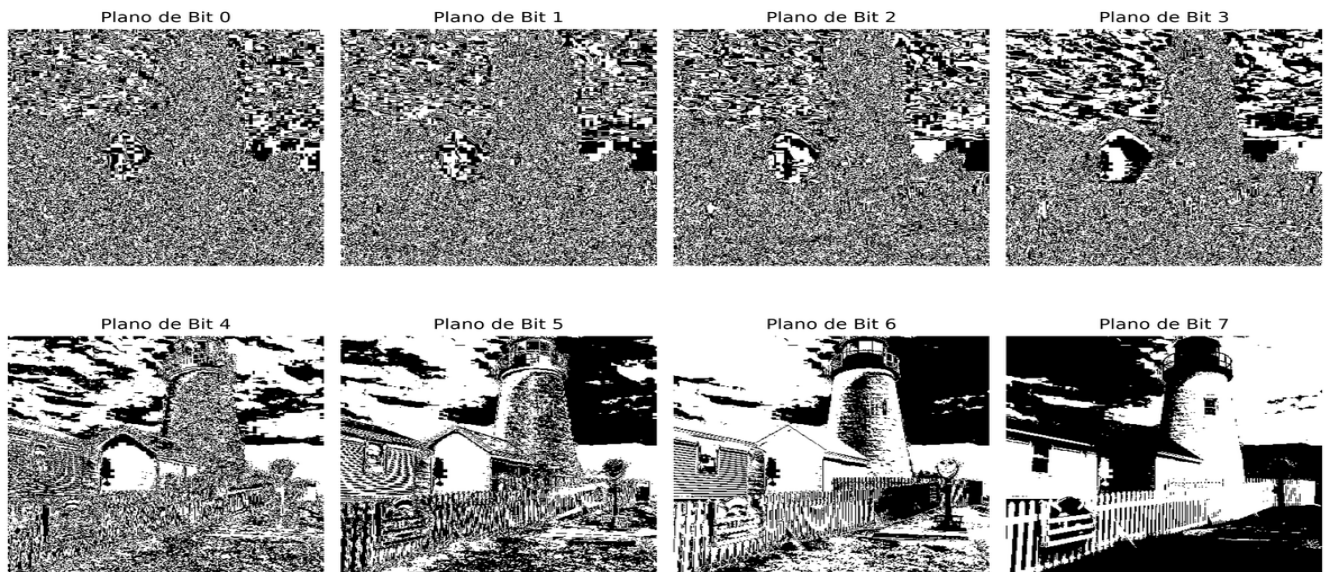


Figura: Visualização dos 8 Planos de Bits extraídos da imagem de teste (Plano 0 = Ruído LSB até Plano 7 = Estrutura MSB).

6. Resultados Experimentais e Benchmarking Detalhado

Abaixo são apresentados os dados empíricos coletados através da suíte automatizada run_all.py (janela de recalibração = 1000 símbolos). Todos os testes foram validados com decodificação completa e verificação bit-a-bit contra o arquivo original (100% de correspondência).

Tabela 1: Desempenho nas Sequências 1D (Entradas de Teste 1 a 8)

Arquivo	Símbolos	Blocos	Overhead	Estado	Bitstream	Total TX	Taxa (%)	T_Enc	T_Dec	Status
INPUT_1	248	1	24 b	13 b	0 b	37 b	85.08%	0.6 ms	0.3 ms	✓ 100%
INPUT_2	248	1	24 b	13 b	2 b	39 b	84.27%	0.4 ms	0.3 ms	✓ 100%
INPUT_3	224	1	24 b	13 b	23 b	60 b	73.21%	0.4 ms	0.2 ms	✓ 100%
INPUT_4	224	1	24 b	13 b	24 b	61 b	72.77%	0.4 ms	0.2 ms	✓ 100%
INPUT_5	264	1	24 b	13 b	46 b	83 b	68.56%	0.6 ms	0.4 ms	✓ 100%
INPUT_6	345	1	24 b	13 b	22 b	59 b	82.90%	0.7 ms	0.7 ms	✓ 100%
INPUT_7	345	1	24 b	13 b	18 b	55 b	84.06%	0.6 ms	0.2 ms	✓ 100%
INPUT_8	938	1	24 b	13 b	96 b	133 b	85.82%	0.5 ms	0.5 ms	✓ 100%
GLOBAL 1D	2.836	8	192 b	104 b	231 b	527 b	81.42%	—	—	✓ 100%

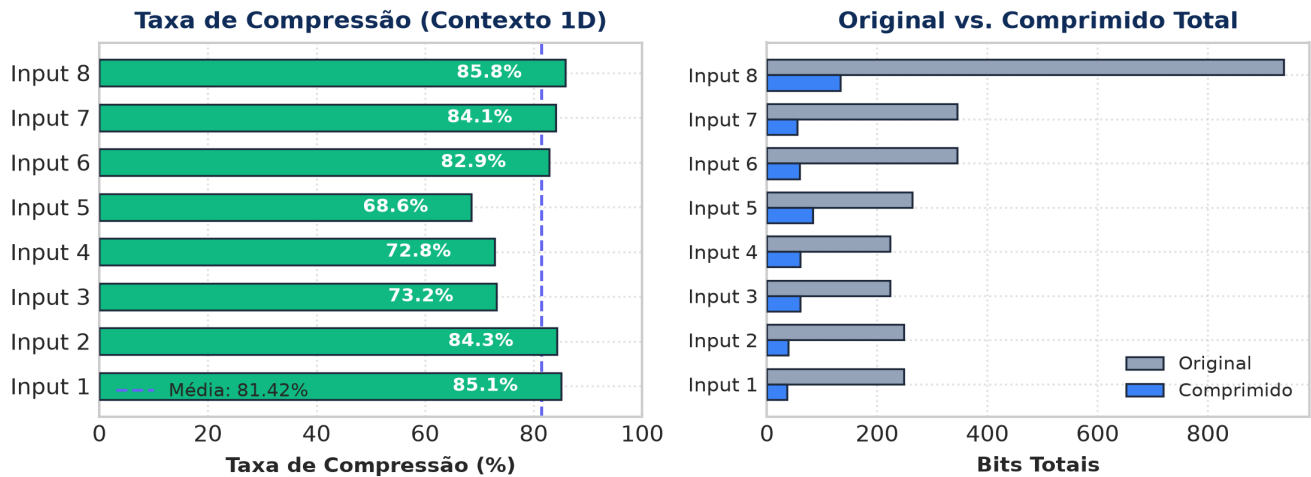
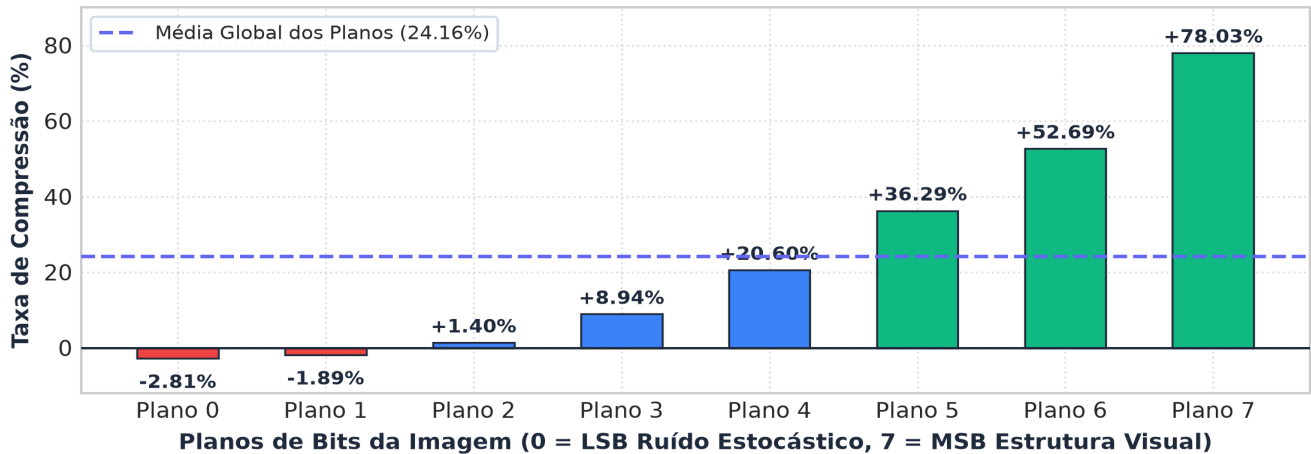


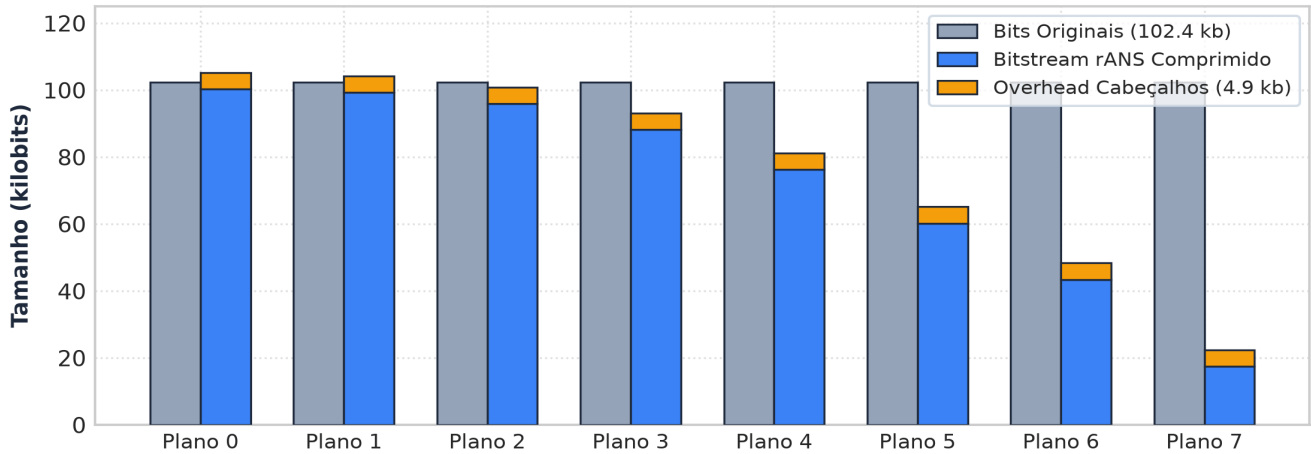
Tabela 2: Desempenho nos Planos de Bits 2D (Imagem 320×320 = 102.400 bits/plano)

Plano	Original	Bloco s	Overhead	Bitstream	Total TX	Taxa (%)	T_Enc	T_Dec	Integr idade
PLANO_0 (LSB)	102.400	103	4.944 b	100.316 b	105.273 b	-2.81%	62.0 ms	56.6 ms	✓ 100%
PLANO_1	102.400	103	4.944 b	99.381 b	104.338 b	-1.89%	59.5 ms	59.8 ms	✓ 100%
PLANO_2	102.400	103	4.944 b	96.010 b	100.967 b	+1.40%	66.4 ms	61.1 ms	✓ 100%
PLANO_3	102.400	103	4.944 b	88.284 b	93.241 b	+8.94%	70.7 ms	64.0 ms	✓ 100%
PLANO_4	102.400	103	4.944 b	76.347 b	81.304 b	+20.60%	68.8 ms	62.7 ms	✓ 100%
PLANO_5	102.400	103	4.944 b	60.284 b	65.241 b	+36.29%	67.3 ms	59.0 ms	✓ 100%
PLANO_6	102.400	103	4.944 b	43.493 b	48.450 b	+52.69%	66.3 ms	62.3 ms	✓ 100%
PLANO_7 (MSB)	102.400	103	4.944 b	17.538 b	22.495 b	+78.03%	61.9 ms	57.4 ms	✓ 100%
GLOBAL 2D	819.200	824	39.552 b	581.653 b	621.309 b	+24.16%	0.52 s	0.48 s	✓ 100%

Taxa de Compressão rANS por Plano de Bits (Contexto 2D Espacial, Bloco=1000)



Decomposição do Volume de Dados Transmitido (kb por Plano de Bit)



7. Guia Prático de Operação e Execução

Todos os módulos do projeto podem ser executados via linha de comando no terminal Linux/macOS/Windows:

Operação Desejada	Comando Terminal (CLI)	Descrição da Saída
Compilar tANS (C)	<code>gcc Encoder_Ans.c -o encoder</code> <code>gcc Decoder.c -o decoder</code>	Gera os executáveis nativos encoder e decoder.
Executar tANS (C)	<code>./encoder input.txt</code> <code>./decoder input_encoded.bin output.txt</code>	Codifica e decodifica o arquivo de bits via FSM estática.
Extrair Bitplanes	<code>python3 Bit_Planes.py</code>	Gera 8 arquivos plano_bit_X.txt e a imagem comparativa.
Codificar 1D (Python)	<code>python3 "ANS-Streaming Encoder.py" --input inputs/input_1.txt --output comp.bin --recalc-window 1000</code>	Executa o rANS com modelo de contexto lookahead 1D.
Codificar 2D (Python)	<code>python3 "ANS-Streaming Encoder.py" --input plano_bit_7.txt --output p7.bin --width 320 --recalc-window 1000</code>	Ativa o modelo espacial 2D de 4 contextos com largura de imagem.
Decodificar & Validar	<code>python3 ANS-Streaming-Decoder.py --encoded p7.bin --output dec.txt --verify plano_bit_7.txt</code>	Decodifica em LIFO e valida identidade exata de 100%.
Bateria Completa	<code>python3 run_all.py --recalc-window 1000</code>	Executa todo o pipeline 1D e 2D e imprime tabela comparativa consolidada.

8. Conclusão e Oportunidades de Evolução

O projeto atingiu com êxito todos os objetivos propostos para a exploração teórica e prática dos Sistemas de Numeração Assimétrica:

- 1. Validação Teórica e Prática:** Comprovou-se a superioridade do ANS sobre Huffman clássico na capacidade de alocar frações de bits, atingindo taxas globais de **81.42%** em sequências com forte correlação linear e **78.03%** em planos estruturais 2D.
- 2. Eficiência de Overhead:** A implementação do **Estado Contínuo** eliminou os gargalos de flush por bloco, mantendo o overhead estrutural abaixo de 5% do volume total de dados.
- 3. Integridade Perfeita:** A reversibilidade simétrica da pilha LIFO assegurou **100% de precisão lossless** em todas as baterias de testes.

Direções para Trabalhos Futuros:

- Vetorização SIMD / C++:** Portar a lógica rANS 2D para extensões AVX-512 ou GPU (CUDA) para processamento em tempo real de fluxos de vídeo 4K.
- Contextos de Ordem Superior:** Expandir a vizinhança 2D para 3 vizinhos (superior, esquerdo e diagonal) totalizando 8 contextos (2^3), aumentando a capacidade de predição em texturas complexas.
- Bypass de Planos Incompressíveis:** Adicionar uma flag de bloco cru (raw/uncompressed) para planos de alta entropia (como Plano 0), evitando qualquer taxa negativa de compressão.